
Problem 8.22.

Part a. Let $ADD = \{\langle x, y, z\rangle \mid x, y, z > 0 \text{ are binary integers and } x + y = z\}$.
Show that $ADD \in$ L.

Proof. To show that $ADD \in$ L, we give a log space algorithm M that decides ADD.

$M =$ "On input $\langle x, y, z\rangle$, where x, y and z are binary integers greater than 0:

1. Let i, n_x, n_y, n_z and m be binary integers, and s and c be binary digits on work tape.
2. Let $n_x = |x|$, $n_y = |y|$, $n_z = |z|$, and m be maximum of n_x and n_y.
3. Let $i = 1$, and $c = 0$.
4. Repeat for m steps:
5. If z does not have i^{th} digit ($i > n_z$), then *reject*.
6. Let s be the sum of c and i^{th} binary digits of x and y. If either x or y does not have i^{th} binary digit, then assume the digit to be 0.
7. If the addition produces a carry, then $c = 1$. Otherwise, $c = 0$.
8. If s and i^{th} digit of z are different, then *reject*.
9. Increment i by 1.
10. If $c = 1$, then:
11. If z does not have i^{th} digit, then *reject*.
12. If c and i^{th} digit of z are different, then *reject*.
13. If z has $(i + 1)^{th}$ digit, then *reject*.
14. Otherwise, if z has i^{th} digit, then *reject*.
15. *Accept*."

□

Part b. Let $PAL\text{-}ADD = \{\langle x, y\rangle \mid x, y > 0$ are binary integers where $x + y$ is an integer whose binary representation is a palindrome$\}$. (Note that the binary representation of the sum is assumed not to have leading zeros. A palindrome is a string that equals its reverse.)
Show that $PAL - ADD \in$ L.

Proof. To show that $PAL\text{-}ADD \in$ L, we give a log space algorithm M that decides $PAL\text{-}ADD$. Let

$$CARRY = \{\langle x, y\rangle \mid x, y > 0 \text{ are binary integers whose sum produces a carry}\}.$$

and

$$BIT = \{\langle x, y, i\rangle \mid x, y, i > 0 \text{ are binary integers, and } i^{th} \text{ digit of } x + y \text{ is } 1\}.$$

The algorithm M uses log space algorithms C and B that decide $CARRY$ and BIT respectively.

$M =$ "On input $\langle x, y\rangle$, where x and y are binary integers greater than 0:

1. Let i, j and m be binary integers on work tape.
2. Let m be the maximum of $|x|$ and $|y|$.
3. Run C on input $\langle x, y\rangle$. If C accepts then $m = m + 1$.
4. Let $i = 1$, and $j = m$.
5. Repeat for $m - 1$ steps:
6. Run B on $\langle x, y, i\rangle$. If B accepts, then $b_i = 1$. Otherwise $b_i = 0$.
7. Run B on $\langle x, y, j\rangle$. If B accepts, then $b_j = 1$. Otherwise $b_j = 0$.
8. If b_i and b_j are different, then *reject*.
9. Increment i by 1, and decrement j by 1.
10. *Accept*."

Next, we give implementation details of algorithms C and B.

$C =$ "On input $\langle x, y\rangle$, where x and y are binary integers greater than 0:

1. Let i and m be binary integers on work tape where $i = 1$ and $m = max(n_x, n_y)$.
2. Let s and c be binary digits on work tape where $s,\ c = 0$.
3. Repeat for m steps:
4. Let s be the sum of c and i^{th} binary digits of x and y. If either x or y does not have i^{th} binary digit, then the digit is 0.
5. If the addition produces a carry, then $c = 1$. Otherwise, $c = 0$.
6. Increment i by 1.
7. If $c = 1$ then *accept*. Otherwise, *reject*."

$B =$ "On input $\langle x, y, b \rangle$, where x, y and b are binary integers greater than 0:

1. Let i and m be binary integers on work tape where m is maximum of $|x|$ and $|y|$.
2. Let s and c be binary digits on work tape where $s,\ c = 0$.
3. Repeat for each $i = 1, \ldots, m$:
4. Let s be the sum of c and i^{th} binary digits of x and y. If either x or y does not have i^{th} digit, then the digit is 0.
5. If the addition produces a carry, then $c = 1$. Otherwise, $c = 0$.
6. If $i = b$:
7. If $s = 1$, then *accept*. Otherwise, *reject*.
8. If $c = 1$ and $b = (m + 1)$, then *accept*.
9. *Reject*."

□